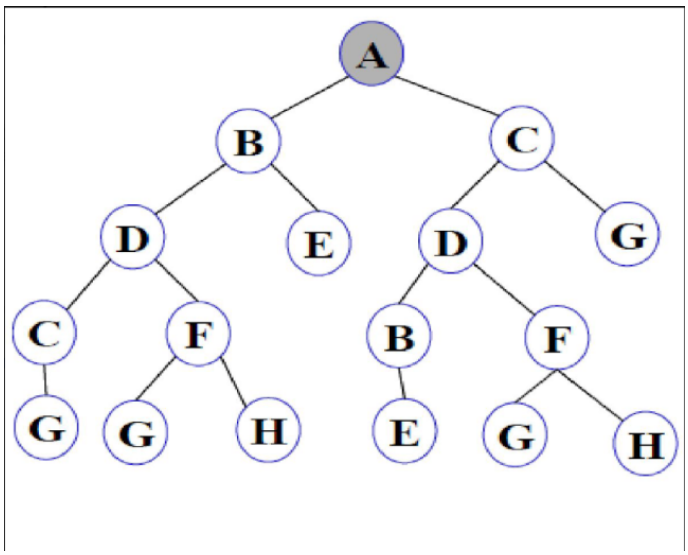
Name: V.Mahalakshmi
Reg No: 192521181
Subject: Artificial Intelligence
Subject Code: CSA1718

BFS and DFS on the Given Search Tree

Aim

To show how Breadth First Search (BFS) and Depth First Search (DFS) work on the given search tree, with A as the start node.

The search tree



Children of each node (left to right):

A: B, C
B (left): D, E
C (right): D, G
D (under B): C, F
D (under C): B, F
C (under D): G
B (under D): E
F (both): G, H
E, G, H: leaf nodes (no children)

Since this is a search tree, a letter can appear many times (for example D, F and G). Each appearance is a separate node of the tree. The tree has 17 nodes and a depth of 4.

Assumptions

Children are expanded from left to right. The goal node is taken as G, since the figure does not mark a goal.

Breadth First Search (BFS)

BFS uses a queue (first in, first out). The shallowest node in the queue is expanded first, so the tree is explored level by level.

Steps (queue shown front to back):

Start: queue = [A]
Visit A, add B, C: [B, C]
Visit B, add D, E: [C, D, E]
Visit C, add D, G: [D, E, D, G]
Visit D (under B), add C, F: [E, D, G, C, F]
Visit E (leaf): [D, G, C, F]
Visit D (under C), add B, F: [G, C, F, B, F]
Visit G (depth 2). This is the goal, so BFS stops here when searching for G.

Without a goal (full traversal), the remaining steps are:

Visit C, add G: [F, B, F, G]
Visit F, add G, H: [B, F, G, G, H]
Visit B, add E: [F, G, G, H, E]
Visit F, add G, H: [G, G, H, E, G, H]
Visit the remaining leaves G, G, H, E, G, H. The queue is empty.

Full BFS order: A, B, C, D, E, D, G, C, F, B, F, G, G, H, E, G, H
Path found to goal G: A - C - G (depth 2, 7 nodes visited)

Depth First Search (DFS)

DFS uses a stack (last in, first out). The deepest node is expanded first, going down one branch as far as possible before backtracking. Children are pushed right to left so that the left child is taken first.

Steps (stack shown bottom to top):

Start: stack = [A]
Pop A, push C, B: [C, B]
Pop B, push E, D: [C, E, D]
Pop D (under B), push F, C: [C, E, F, C]
Pop C, push G: [C, E, F, G]
Pop G (depth 4). This is the goal, so DFS stops here when searching for G.

Without a goal (full traversal), the remaining steps are:

Pop F, push H, G: [C, E, H, G]
Pop G, then H (both leaves): [C, E]
Pop E (leaf): [C]. The whole left subtree of A is finished, so DFS backtracks.
Pop C (right child of A), push G, D: [G, D]
Pop D, push F, B: [G, F, B]
Pop B, push E: [G, F, E]
Pop E (leaf): [G, F]

Pop F, push H, G: [G, H, G]
Pop G, H, G. The stack is empty.

Full DFS order: A, B, D, C, G, F, G, H, E, C, D, B, E, F, G, H, G
Path found to goal G: A - B - D - C - G (depth 4, 5 nodes visited)

Comparison

Data structure: BFS uses a queue, DFS uses a stack.
Order: BFS goes level by level, DFS goes deep along one branch first.
Goal G: BFS reaches G at depth 2 (A - C - G) after visiting 7 nodes. DFS reaches G at depth 4 (A - B - D - C - G) after visiting 5 nodes.
Shortest path: BFS always finds the shallowest goal, so its path is the shortest. DFS found a longer path, because it went down the left branch first.
Memory: BFS must keep a whole level in the queue, so it needs more memory. DFS keeps only the current branch and its pending siblings.

Result

Both searches visit all 17 nodes if no goal is given, but in different orders. BFS gives the shortest path to G (A - C - G), while DFS reaches G sooner in terms of nodes visited but along a longer path (A - B - D - C - G).